Modeling Distributed Information: Send+More=Money

Andrés F. Barco

Department of Electronics and Computing Science
Pontificia Universidad Javeriana - Cali
Cali, Colombia, South America
anfelbar@javerianacali.edu.co

Abstract. Concurrent behavior is present in most information and communication technologies, from Web Services to Social Networks and Cloud Computing. For these systems, the knowledge representation of the involved agents are of crucial importance for an accurate description and modeling of their behavior. Further, the distributed nature of information forces to analyze system constraints in both processing and storage capabilities. On this regard, we present simple implementations of the send+more=money puzzle using several scenarios in which the information is distributed among a set of agents that may know certain information while ignoring other. We use a constraint-based interpreter to model the puzzle according with spatial and epistemic specifications. We use three well-know examples to illustrate agent interaction.

Introduction

Concurrent behavior is present in most information and communication technologies, from Web Services to Social Networks and Cloud Computing. For these systems, the knowledge representation of the involved agents are of crucial importance for an accurate description and modeling of their behavior. Indeed, reasoning about other agents' knowledge is a fundamental aspect on behavioral approaches, e.g., game theory [10], artificial intelligence [12] and logic [3]. In each scenario, see [3, 7] for some examples, where epistemic interactions takes place, the sharing of knowledge, e.g., commercial strategies, connections, logins and passwords, allows different properties to emerge.

On this regard, we present simple implementations of the send+more=money puzzle using several scenarios in which the information is distributed among a set of agents that may know certain information while ignoring other. Special attention should be paid to the representation of knowledge and distributed information. We use a constraint-based interpreter [4] to model the puzzle according with spatial and epistemic specifications. The interpreter is based on two novel process calculi that use spatial and epistemic modal operators as its underlying logic [11]. In consequence, the interpreter allow to play with agents that have different processing and storage constraints. The distributed nature of information enables a large family of problems to be specified in the calculi and

to be simulated by the interpreter. The addressed puzzle shows the potential expressive power of the calculi and its modeling benefits. Along the paper we use three well-know examples to illustrate agent interaction.

The paper is structured as follows. The section 1 is dedicated to a brief description of the puzzle, the spatial and epistemic ccp calculi and the tool we use. Next, we describe the notion of inconsistency confinement which reinforces the spatial model. Section 3 illustrates the concept of distributed knowledge among a set of rational agents. We present the implementation of a spatial and epistemic send+more=money in section 4. The last section is dedicated to some remarks and related work. Finally, a bibliography is included.

1 Preliminaries

In this section we present puzzle specification along with the mathematical model and the programming environment we use for the implementation. As the paper does not focus on the model nor the tool but in modeling, the read interested in more information should look it in the references. The reader should know, however, that it is not mandatory to deepen in the mathematical foundations, a basic understanding of constraint programming will suffice.

1.1 The puzzle

The well-known puzzle send+more=money was created almost a century ago. The man who created it was Henry Dudeney [6], a recognized mathematician of his time. Dudeney argued that the simplest puzzle should not be passed over without careful attention. He considered in particular arithmetic problems, like the send+more=money, to be the most interesting ones. Indeed, mathematicians and logicians reason about the underlying true behind the puzzles using different representation and intuitions. Nonetheless, although the send+more=money puzzle was not conceived to be studied within a particular model, it seems that the the problem fits neatly with the declarative nature of the constraint programming paradigm. The basic description of the puzzle is shown next. The specification is taken from the Oz Programming Interface webpage [1].

Send+more=money The Send More Money problem consists of finding distinct digits for the letters D, E, M, N, O, R, S, Y such that S and M are different from zero (no leading zeros) and the equation SEND+ MORE=MONEY is satisfied. The unique solution of the problem is 9567+1085= 10652.

1.2 Spatial and epistemic concurrent constraint calculi

The traditional concurrent constraint programming viewpoint is not well equipped for the modeling of distributed information. This is due to the local computing constraints agents may have, i.e., both local processing and storage. Spatial ccp

and Epistemic ccp [11] extend concurrent constraint programming calculus in order to capture some notions of spatiality and knowledge that have not been addressed by other extensions. We encourage the reader to study the theoretical model created by Knight et al. as described in [11] to get a better understanding of the calculi. In the following, we describe the language of constructions terms.

Let P and Q be two spatial-epistemic ccp processes. The language of construction terms as:

$$P, Q := \mid tell(c) \mid ask(c) \rightarrow P \mid P \parallel Q \mid [P]_i \mid X$$

Assuming that for each process variable X there exists a process definition, possibly recursive, of the form $X \stackrel{def}{=} P$.

Intuitively, each agent i has his own local store $[\cdot]_i$ where processes and other agents' stores may reside. Consequently, the spatial construct $[P]_i$ represents the process P within the store of agent i. Ask and parallel semantics remains as in classic ccp. However, the application of tell semantics has two different meaning.

Essentially, a tell operation may be seen as spatial or epistemic. Used as a spatial operator, a tell only adds partial information to the agent's store. This is akin to the notion of belief; agent i may believe that it is raining while agent j believes it is not. Consequently, no spatial tell operation can cause the overall computation to fail, i.e., $[tell(false)]_i \neq false$ and $[tell(c)]_i \sqcup [tell(d)]_j \neq false$ even when $c \sqcup d = false$[1]. On the other hand, tell operator in its epistemic forms allows to represent facts. Thus, $[tell(c)]_i \sqcup [tell(d)]_j = false$ when $c \sqcup d = false$. Intuitively, the process $[tell(c)]_i$ is that c is added to the knowledge of agent i. Some interesting properties of the epistemic tell operator are that; a) after $[tell(c)]_i$ is executed, the store entails c (for any constraint c); meaning that if an agent knows something then it is true; b) $[tell(c)]_i$ is idempotent; meaning that agent i knows that he knows c; and c) after $[[tell(c)]_j]_i$ is executed, c will be in i's store. This is because if i knows that he knows c, he can conclude c from this fact. The details of spatial and epistemic constraint systems, along with their operational and denotational semantics, can be found in [11].

1.3 K-stores: Sccp and Eccp Interpreter

In essence, an interpreter is a computer program that executes expressions of a particular programming language. Such program, often called the *defined* language, is built using other programming language, also called the *defining* language [13]. An useful characterization for programming languages is related with the underlying logic the language is based on. Most programming languages and interpreters are based in the well-studied logic of the λ-calculus [2]. This calculus is universal in the sequential computation, i.e., any computable function can be expressed using the calculus. However, most of real-life system do not exhibit a sequential behavior but a concurrent and reactive one. Thus, using the λ-calculus for the modeling of concurrent-reactive systems is not likely.

[1] The symbol $\sqcup$, least upper bound, represent the join of information.

The `K-stores` interpreter is a Prolog implementation of the operational semantics of the spatial and epistemic ccp calculi allowing the programmer to simulate distributed information systems [4]. Its main feature consists of an implementation of a spatial (distributed) store that allows epistemic information in it. In particular, it implements the S4 epistemic logic axiomatic system [3, 9]. The system supports the specification of (named) processes along with the ccp classic primitives, namely, ask and tell operations. The interpreter is built with the SWI-Prolog programming environment [15] and uses its constraint logic programming module as its underlying constraint system. For further information about this tool refer to [4].

2 Important Properties

Distributed Knowledge `K-stores` allows the programmer to simulate real-life scenarios. Scenarios are characterized by having spatial properties or epistemic ones. Perhaps, a given system exhibits spatial and epistemic properties at the same time, however, our decision is to define programs specified either epistemically or spatially.

It is worth noting the difference between spatial and epistemic information. If an agent makes a spatial tell operation in his own store, the information added does not affect other agent's stores. On the contrary, an epistemic tell will, potentially, change other stores. For instance, if the agent A knows that agent B knows ϕ, then B must know ϕ. This is due that epistemic structure S4 does not allow people to know false statements [3, 9]. Conversely, if agent A knows that X is equal to 0 and agent B knows that X is equal to 1, an incosistency is raised.

In this section we give some intuition about possibles scenarios in a give problem. Such intuition will allow us to describe the distributed information that may be present in a system. We use the example called *The logicians* [7] because it makes evident the distributed nature of information and that agents can gain information independently of each other. In scenarios such as this, any given agent has his own processing and storage constraints. Consequently, all agents, represented with different stores, use a particular (possibly) disjoint set of constraints to draw its own conclusions.

The logicians The logicians Alice and Bob are sitting in their windowless office wondering whether or not it is raining outside. Now, none of them actually knows, but Alice knows something about her friend Carol, namely that Carol wears her red coat only if it is raining. Bob does not know this, but he just saw Carol, and noticed that she was wearing her red coat.

The fact that is raining can become knowledge for Alice and Bob if they make the right assumptions and question. This is due that the fact "it is raining" is distributed knowledge among Alice and Bob. In short, an event ϕ is distributed knowledge among a group of agents G if and only if ϕ follows from the knowledge intersection of all individual agents in G. Semantically, ϕ is distributed knowledge

among G if and only if ϕ is true in all worlds that every agent in G considers possible [14]. Distributed knowledge can be modeled as $D_G(\phi) = \bigwedge_{i \in G} K_i$

Where $K_i\phi$ means the event that "agent i knows ϕ". The result of the operator K over an agent can be seen as a projection over the state of possible worlds that shapes the agent behavior. This notion is found materialized in most distributed information systems as social networks.

Isolation One of the most important intuitions of the spatial ccp calculus is the isolation of information, also referred to inconsistency confinement in [11]. A graphical representation may help us understand the relevance of store isolation to represent agents' knowledge. In particular we want to reason about agents that reason about other agent's knowledge. The next example, taken from [5], and its representation shows us the importance of first order knowledge and higher order knowledge. The graphic makes clear that knowledge can be modeled using the constraint programming paradigm which is one of our goals.

Hats puzzle Three people, Adam, Ben and Clark are siting in such a way that Adam can see Ben and Clark, Ben can see Clark and Clark can see no one. Without seeing the color, a white hat or a red hat is put in the head of each agent. Suppose that all three hats are red. Then, with open eyes, they are asked whether they know what the colors of their hats are. In this setup all agents answers are negative. However, when an agent makes the next announcement "there is at least one of you wearing a red hat" the result is quite different. They are asked again which color they are wearing. Then Adam answers that he does not know. Then Ben answers that he does not know either. Finally Clark says that he knows the color of his hat. What color is Clark's hat?

The possible state of knowledge for the puzzle, shown in fig 1, may be represented[2] as a set of vertices [9]. Each corner of the cube is defined by three values. All of them may be either `W` or `R` representing whether the hat of the agent i is white or red, respectively. Each of those vertices in the cube models a given state of affairs in the agents' minds. Thus, the first cube is the moment when no body has answer the questions; they think that any combination of hats' colors is possible.

An agent is sure about his hat color in a given state, if none of their vertices connects with another vertex in which the color of his hat is different. It is worth noting that Clark's knowledge is unambiguous; Clark knows that his hat is actually red. Public announcements of his two friends enabled Clark to find the answer. Distributed knowledge is generated only when Adam and Ben answer the question aloud. We leave the reasoning of this puzzle to the reader.

[2] The graphical representation is an adaptation of one presented in [9].

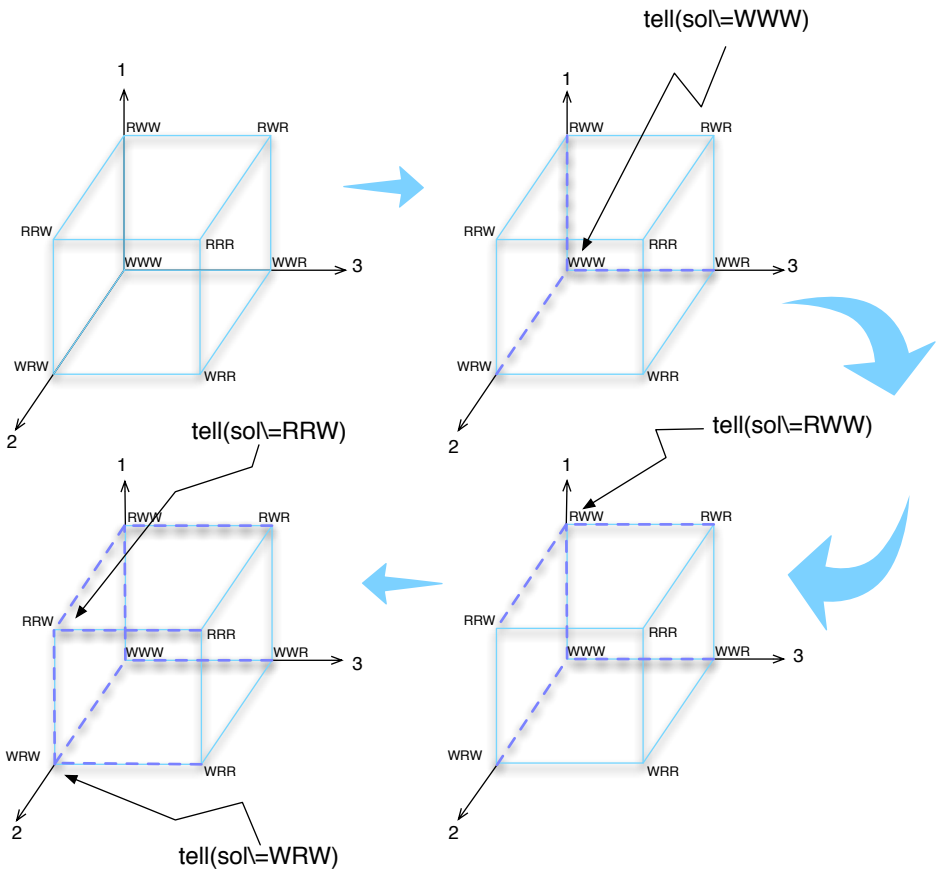

Fig. 1. Evolution of the puzzle graphical representation.

3 Spatial and epistemic send+more=money

Bearing in mind the send+more=money puzzle we describe above, we create program specifications of the puzzle involving spatial hierarchies and knowledge. The distributed nature of the spatial and epistemic ccp language implies that any agent is running in a separate computer node or processor core. Although the interpreter does not allow execution on different nodes, it is easy to map the specification in a programming environment that allows parallel execution of programs. In addition, in order to understand the programs, we need to keep in mind that any tell operation with an agent as argument translates into a tell operartion in that agent store (this resembles post operations in social networks) and that the semantics of askI is asking some information in the owned (nested) store (see [4] for further details).

Scenario 1: The first attempt to solve the problem uses a spatial division. Two agents are trying to reach a solution. Agent `katherine` ignores that all variables are pairwise distinct, whereas agent `andrew` does not know that `s` and `m` can not be zero.

```
parallel([
  spatial(katherine, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
              #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell( M #\= 0), tell(S #\= 0),
    tell(solve([S,E,N,D,M,O,R,Y]))]),
  spatial(andrew, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
              #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell(all_different([S,E,N,D,M,O,R,Y])),
    tell(solve([S,E,N,D,M,O,R,Y]))])
]).

Knowledge for agent katherine --> {9,0,0,0,1,0,0,0}
Knowledge for agent andrew --> {2,8,1,7,0,3,6,5}
```

In this specification, agents have different information about the state of affairs, i.e., the complete set of constraints. Thus, neither agent is able to reach the valid answer.

Scenario 2: In the second specification the agent `katherine` gives information (post) to `andrew` about the constraints she knows. The only new information is that `m` can not be zero. Thus, `andrew` reach a new and valid conclusion.

```
parallel([
  spatial(katherine, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
              #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell(M #\= 0),
    tell(andrew, tell(M#\=0)),
    tell(solve([S,E,N,D,M,O,R,Y]))]),
  spatial(andrew, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
              #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell(all_different([S,E,N,D,M,O,R,Y])),
    tell(solve([S,E,N,D,M,O,R,Y]))])
]).

Knowledge for agent katherine --> {9,0,0,0,1,0,0,0}
Knowledge for agent andrew --> {9,5,6,7,1,0,8,2}
```

Scenario 3: The third case presents an agent that has some local representation, nested store, of other agent's knowledge. In this specification, `katherine` beliefs that `andrew` knows certain set of constraint. Using that beliefs, she can reach the valid conclusion.

```
parallel([
  spatial(katherine, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    spatial(andrew, [tell([S,M] ins 0..9), tell( M #\= 0),
        tell(S #\= 0)]),
    askI('andrew:own', M#\=0 -> M#\=0),  askI('andrew:own', S
        #\=0 -> S#\=0),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
          #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell(all_different([S,E,N,D,M,O,R,Y])),
    tell(solve([S,E,N,D,M,O,R,Y]))])
]).

Knowledge for agent katherine --> {9,5,6,7,1,0,8,2}
Knowledge for nested:katherine's representation of andrew -->
     {1..9,1..9}
```

Scenario 4: In this specification, and the next one, we use an epistemic viewpoint to solve the problem. In this case, both `katherine` and `andrew` reach different assignments for the variables. Thus, an inconsistency among stores is raised.

```
parallel([
  epistemic(katherine, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
              #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell( M #\= 0), tell(S #\= 0),
    tell(solve([S,E,N,D,M,O,R,Y]))]),
  epistemic(andrew, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
              #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell(all_different([S,E,N,D,M,O,R,Y])),
    tell(solve([S,E,N,D,M,O,R,Y]))])
]).

Global store failed: inconsistency among stores
aborting execution...
```

Scenario 5: Finally, this epistemic scenario models the case where `katherine` has a local epistemic representation of `andrew`. The information in it, the fact that all variables are pairwise distinct, enables `katherine` to find the valid answer to the puzzle.

```
parallel([
  epistemic(katherine, [tell([S,E,N,D,M,O,R,Y] ins 0..9),
    epistemic(andrew, [tell([S,M] ins 0..9), tell(
        all_different([S,E,N,D,M,O,R,Y]))]),
    tell(sum([S*1000, E*100, N*10, D*1, M*1000, O*100, R*10,
        E*1],
          #=, [M*10000, O*1000, N*100, E*10, Y*1])),
    tell( M #\= 0), tell(S #\= 0),
    tell(solve([S,E,N,D,M,O,R,Y]))])
]).

Knowledge for agent katherine --> {9,5,6,7,1,0,8,2}
Knowledge for nested:katherine's representation of andrew -->
     {1..9,1..9}
```

4 Conclusions

Social networks, mainly web-based networks, are a growing field of research because of their complexity and ubiquity. In such multi-agent systems information flows in huge magnitudes from client to client. Two fundamental features in these networks are the private data locality and the (possibly constrained) public information posting that is allowed for agents. Furthermore, information exchange may take place within a subset of agents inside the network. Moreover, different information (knowledge) shared inside the network may become common knowledge throughout the entire network (e.g. worldwide disasters).

Such distributed information and knowledge in systems with concurrent behavior are too complex to understand using an unique language or model [8, 7]. Nonetheless, modeling tools , such as the `K-stores` interpreter, can be used to simulate processes that have their own local storage and may gain information by means of a shared medium, such as cloud computing and social networks. We presented some implementations of the send+more=money puzzle using spatial and epistemic interactions. We show how distributed information among a set of agents may reach solutions or inconsistency depending on locality properties, thus, illustrating the modeling properties of the sccp and eccp calculi and the `K-stores` tool. A more elaborated study should include the implementation of the puzzle using various processing nodes.

We acknowledge that in order to solve a distributed problem an agent needs to know all constraints over the variables. Nonetheless, the proposed modeling capabilities are enough to represent evident real life scenarios in which no agent has all information. Moreover, there exists a global store where all information is posted, given the chance to reach an inconsistency or a solution. However, we chose not to search for a solution in that global store because it does not belong to an agent with reasoning capabilities. Instead, we want the global store to help us to respect the epistemic logic axioms. More effort should be put on different views of such global store.

References

1. Oz Programming Interface. `http://mozart-oz.org`. Accessed: 2013-07-10.
2. S. Kamal Abdali. A lambda-calculus model of programming languages - i. simple constructs. *Comput. Lang.*, 1(4):287–301, 1976.
3. Alexandru Baltag, Lawrence S. Moss, and Slawomir Solecki. The logic of public announcements, common knowledge, and private suspicions. In *Proceedings of the 7th conference on Theoretical aspects of rationality and knowledge*, TARK '98, pages 43–56, San Francisco, CA, USA, 1998. Morgan Kaufmann Publishers Inc.
4. A. Barco, S. Knight, and F. Valencia. K-stores an spatial and epistemic concurrent constraint interpreter. In *Functional and Constraint Logic Programming - 21th International Workshop, WFLP 2012, Nagoya, Japan, May 29th, Proceedings*, 2012.
5. Luc Bovens and Wlodek Rabinowicz. The puzzle of the hats. *Synthese*, 172:57–78, 2010. 10.1007/s11229-009-9476-1.
6. Henry Dudeney. Send more money. *Strand Magazine*, pages 68–79, July, 1924.
7. R. Fagin, J. Halpern, Y. Moses, and M. Vardi. *Reasoning About Knowledge*. MIT Press, 1995.
8. H. Garavel. Reflections on the future of concurrency theory in general and process calculi in particular. Research Report RR-6368, INRIA, 2007.
9. J. Geanakoplos. Common knowledge. In *Proceedings of the 4th conference on Theoretical aspects of reasoning about knowledge*, pages 254–315. Morgan Kaufmann Publishers Inc., 1992.
10. Herbert Gintis. Rationality and common knowledge. *Rationality and Society*, 22(3):259–282, August 2010.
11. Sophia Knight, Catuscia Palamidessi, Prakash Panangaden, and Frank Valencia. Spatial and epistemic modalities in constraint-based process calculi. In Maciej Koutny and Irek Ulidowski, editors, *CONCUR 2012 – Concurrency Theory*, volume 7454 of *Lecture Notes in Computer Science*, pages 317–332. Springer Berlin Heidelberg, 2012.
12. Marc Ponsen, Pieter Spronck, Héctor Muñoz Avila, and David W. Aha. Knowledge acquisition for adaptive game ai. *Sci. Comput. Program.*, 67:59–75, June 2007.
13. John C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, volume 2 of *ACM '72*, pages 717–740, New York, NY, USA, 1972.
14. Floris Roelofsen. Distributed knowledge. *Journal of Applied Non-Classical Logics*, 17(2):255–273, 2007.
15. Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. Swi-prolog. *Computing Research Repository*, abs/1011.5332, 2010.